

Toy Model II: Kernel Regression

Mark Rhee

Chapter 3

In the previous chapter we studied deep linear networks, i.e., networks that are a linear function of their input data. What made the analysis interesting was their nonlinear training dynamics. Such nonlinear training dynamics are a result of a subtlety in the word "*linear*" in deep linear network. It's true that the network is linear with respect to each individual weight matrix W_i . However, if we consider the weights collectively, stacking all the scalar weights in one ginormous *parameter vector* θ , then a deep linear network is decidedly *not* linear in θ . In fact, a deep linear network with L layers is an L th order polynomial of θ . This is essentially what results in coupled dynamics and why we had to solve nonlinear ODEs to obtain the optimization trajectories of the weights. This is also what made deep linear networks a decent toy model of neural network training dynamics.

In this chapter, instead of networks that are a linear function of their input data, we will study networks that are approximately a linear¹ function of their parameter vector θ . A good way to think about such networks is that their weights θ deviate so little from initialization throughout training that the network as a whole is well approximated by its first-order (linear) Taylor expansion centered at its weights at initialization $\theta^{(0)}$:

$$\hat{f}^{(t)}(x) \approx \hat{f}_{\text{lin}}^{(t)}(t) := \hat{f}^{(0)}(x) + \left(\nabla_{\theta} \hat{f}(x) |_{\theta=\theta^{(0)}} \right)^{\top} (\theta^{(t)} - \theta^{(0)}) \quad (1)$$

$$= \text{const.} + (\text{const. vector})^{\top} (\text{change in weights}) \quad (2)$$

for all $t \geq 0$. Crucially, the terms $\hat{f}^{(0)}(x)$ and $\nabla_{\theta} \hat{f}(x) |_{\theta=\theta^{(0)}}$ are determined at initialization and stay constant throughout training. The weights of this kind of network undergo very simple, exactly solvable optimization trajectories. For neural networks in the infinite-width limit, there exist hyperparameter settings that result in this kind of linear training.² Because the weights undergoing this kind of training barely move, we say that such hyperparameter settings put the network in the *lazy* regime. It's also worth noting that a recent paper from Meterez et al. (2026)³ found that for LLMs, such a first-order Taylor expansion well-approximates the full model for meaningful windows of time during pretraining.

1 Lazy training in the infinite-width limit

When dealing with coupled systems that have many moving parts, physicists love to take some parameter(s) to infinity and see if things simplify. A classic example of this technique is the thermodynamic limit. The key idea is that macroscopic behavior becomes stable and easier to analyze when micro-level fluctuations vanish. Might something like that be possible for neural networks?

¹Technically affine.

²There are also hyperparameter settings that result in non-lazy training in the infinite-width limit. More on that in the next chapter.

³A Defense of the Quadratic Model, <https://arxiv.org/pdf/2607.21716>

Consider a 2-layer neural network:

$$\hat{f}(x) = W_2 \sigma(W_1 x). \quad (3)$$

We can rewrite this in a suggestable way as follows:

$$\hat{f}(x) = \sum_{i=1}^H u_i \sigma(w_i^\top x) \quad (4)$$

where the $u_i \in \mathbb{R}^{n_{\text{out}}}$ are columns of $W_2 \in \mathbb{R}^{n_{\text{out}} \times H}$ and the $w_i \in \mathbb{R}^{n_{\text{in}}}$ are rows of $W_1 \in \mathbb{R}^{H \times n_{\text{in}}}$. Written this way, it's clear that a 2-layer neural network is just an input-dependent linear combination of output vectors u_i . Furthermore, at initialization, the pairs (u_i, w_i) are drawn independently across i from some fixed distribution. This means that for any fixed input x , the H summands $u_i \sigma(w_i^\top x) =: X_i$ are themselves i.i.d. random vectors in $\mathbb{R}^{n_{\text{out}}}$. In other words,

$$\hat{f}(x) = \sum_{i=1}^H X_i \quad (5)$$

is precisely a sum of H i.i.d. random vectors at initialization. This is exactly the kind of object the central limit theorem (CLT) governs.

To cleanly make a CLT-style concentration argument, we need to send the number of summands to infinity, i.e., $H \rightarrow \infty$. However, there's a problem with absent-mindedly taking this infinite-width limit: the value of the function $\hat{f}(x)$ will also go to infinity! One way to prevent $\hat{f}(x)$ from blowing up is to scale the output inversely proportional to the size of the width H . Such output scaling allows us to initialize weights as standard Gaussians ($u_{ij} \sim \mathcal{N}(0, 1)$ and $w_{ij} \sim \mathcal{N}(0, 1)$) and use a width-independent learning rate η . By and large, there are two classical ways of scaling a sum of i.i.d. random variables: a $\frac{1}{H}$ scaling and a $\frac{1}{\sqrt{H}}$ scaling. The former is associated with the law of large numbers (LLN) while the latter is associated with the central limit theorem (CLT). The latter is what leads to lazy training, so we'll study that first:

$$\hat{f}^{(t)}(x) = \frac{1}{\sqrt{H}} \sum_{i=1}^H u_i^{(t)} \sigma(w_i^{(t)\top} x), \quad (6)$$

where $u_{ij}^{(0)} \sim \mathcal{N}(0, 1)$ and $w_{ij}^{(0)} \sim \mathcal{N}(0, 1)$ and the learning rate η is some small width-independent number (e.g., 0.1). We'll see that this setup not only allows us to leverage the CLT at initialization, it also results in the weights moving so little from initialization that for large H , the network $\hat{f}^{(t)}(x)$ is well-approximated by its first-order Taylor expansion $\hat{f}_{\text{lin}}^{(t)}(x)$, i.e., it undergoes lazy training.

Concretely, we'll try to understand why the following statement might be true:

$$\sup_{t \geq 0} \left\| \hat{f}^{(t)}(x) - \hat{f}_{\text{lin}}^{(t)}(x) \right\|_2 = \mathcal{O} \left(\frac{1}{\sqrt{H}} \right). \quad (7)$$

In the $H \rightarrow \infty$ limit, the above equation implies that the trajectory of $\hat{f}^{(t)}(x)$ is exactly the trajectory of $\hat{f}_{\text{lin}}^{(t)}(x)$. To make sense of the above statement, we will now introduce one of the central objects of study in learning mechanics: the neural tangent kernel (NTK).

1.1 The neural tangent kernel

At its core, the neural tangent kernel is a tool for reasoning about networks undergoing gradient descent in *function space* instead of *weight space*. Recall that weight space is the collection of all possible weight configurations. It can be thought of as a high dimensional vector space $\mathbb{R}^M$ where M is the total number of tunable weights. Every point in this space of weights is mapped to a scalar loss value by the loss function $\mathcal{L} : \mathbb{R}^M \rightarrow \mathbb{R}$. These loss values make up the loss landscape. In chapters 1 and 2, we mostly thought about networks' weight space dynamics; i.e., how the weights θ traverse the loss landscape according to gradient flow:

$$\dot{\theta}^{(t)} = -\nabla_{\theta} \mathcal{L}(\theta^{(t)}). \quad (8)$$

However, at the end of the day, the only reason we care about the dynamics of the weights θ is because they tell us about the dynamics of the network function $\hat{f}(x)$ —we want to see how the predictions for each data point $x \in \mathbb{R}^{n_{\text{in}}}$ evolve over time. Could we bypass the middleman θ ⁴ and directly study the dynamics of the time-evolving prediction matrix $\hat{f}^{(t)}(X) \in \mathbb{R}^{P \times n_{\text{out}}}$ (where $X \in \mathbb{R}^{P \times n_{\text{in}}}$ and P is the number of data points in the training set)?

For the rest of this chapter we take $n_{\text{out}} = 1$ so that $\hat{f}(x) \in \mathbb{R}$ and $\hat{f}(X) \in \mathbb{R}^P$ ⁵. It will be convenient to name the Jacobian of the network's predictions with respect to its weights,

$$J^{(t)} := \frac{\partial \hat{f}^{(t)}(X)}{\partial \theta} \in \mathbb{R}^{P \times M}, \quad J_{ik}^{(t)} = \frac{\partial \hat{f}^{(t)}(x_i)}{\partial \theta_k}, \quad (9)$$

whose i th row is the gradient of the network's prediction at x_i with respect to all M weights.

Now apply the chain rule to see how the predictions move:

$$\dot{\hat{f}}^{(t)}(X) = J^{(t)} \dot{\theta}^{(t)} = -J^{(t)} \nabla_{\theta} \mathcal{L} = -\underbrace{J^{(t)} J^{(t)\top}}_{=: K^{(t)}} \nabla_{\hat{f}(X)} \mathcal{L} \left(\hat{f}^{(t)}(X) \right), \quad (10)$$

The matrix that appeared is the *empirical neural tangent kernel*:

$$K^{(t)} := J^{(t)} J^{(t)\top} \in \mathbb{R}^{P \times P}, \quad K_{ij}^{(t)} = \sum_{k=1}^M \frac{\partial \hat{f}^{(t)}(x_i)}{\partial \theta_k} \frac{\partial \hat{f}^{(t)}(x_j)}{\partial \theta_k}. \quad (11)$$

For now, we'll refer to the empirical neural tangent kernel as simply the NTK. The following are three important observations we can make about the NTK. First, $K^{(t)}$ is symmetric and positive semi-definite. Second, it has no information about the loss or the labels: it is built entirely from the architecture and the current weights, and the loss enters Equation (10) only through the separate factor $\nabla_{\hat{f}(X)} \mathcal{L}$. Third, Equation (10) is exact. We have made no approximation, taken no limit, and assumed nothing about the architecture; every neural network trained by gradient flow obeys it. Gradient flow was used only to get a clean ODE, and the definition of $K^{(t)}$ itself is perfectly sound under discrete gradient descent.

⁴There are pros and cons to studying weight space dynamics. On the one hand, it's the most visceral way to think about what's going on during gradient descent. On the other hand, there's a lot of stuff to keep track of and there exist many symmetries in weight space that create annoying redundancies.

⁵Nothing conceptual changes for $n_{\text{out}} > 1$; the kernel simply becomes an $N n_{\text{out}} \times N n_{\text{out}}$ matrix of blocks, one $n_{\text{out}} \times n_{\text{out}}$ block per pair of training points.

1.1.1 Neural networks do a generalized version of linear regression

Take a hard look at Equation (10). Hopefully it looks familiar. It is almost exactly the differential equation for overparameterized linear regression from chapter 1:

$$\dot{\hat{f}}_{\text{linreg}}^{(t)}(X) = X\dot{w}^{(t)} = -XX^\top (Xw^{(t)} - y) \quad (12)$$

$$= -XX^\top \nabla_{\hat{f}_{\text{linreg}}(X)} \mathcal{L}(\hat{f}_{\text{linreg}}^{(t)}(X)). \quad (13)$$

Linear regression is the special case in which the Jacobian is the data matrix itself, $J = X$, so that the kernel is just the matrix of inner products between data points, $K_{ij} = x_i^\top x_j$. Being linear in its weights, linear regression has a Jacobian that does not depend on w at all, which is precisely why its kernel is constant and its dynamics are solvable in closed form.

Define the *feature map*

$$\phi^{(t)}(x) := \frac{\partial \hat{f}^{(t)}(x)}{\partial \theta} \in \mathbb{R}^M, \quad \text{so that} \quad K_{ij}^{(t)} = \langle \phi^{(t)}(x_i), \phi^{(t)}(x_j) \rangle. \quad (14)$$

We can think of the NTK as first projecting each data point into an M -dimensional feature space, then taking inner products there. Thus, a neural network can be thought of as doing linear regression in some high-dimensional feature space using a data-dependent, time-evolving feature map $\phi^{(t)}$! This is a powerful way to think about feature learning.

At this point, it might seem like we've cracked neural networks: they're just a more data-dependent version of linear regression.⁶ A full understanding feels within reach. But this is an illusion. All we've really done is push the problem of feature learning up one level of abstraction; from evolving weights to an evolving kernel. Studying the latter is not necessarily any easier than studying the former. That being said, the NTK isn't a dead end. It gives us a natural place to start: the lazy regime, where the kernel stays (approximately) frozen and the dynamics become tractable (essentially the same dynamics as linear regression). Furthermore, we'll find that there are interesting insights to be gained about generalization from networks in the lazy regime.

For a network that is linear in its weights, such as the linearization $\hat{f}_{\text{lin}}$ from the start of this chapter, the NTK will stay constant throughout training. By construction $\phi_{\text{lin}}^{(t)}(x) = \nabla_\theta \hat{f}(x)|_{\theta=\theta^{(0)}}$ is independent of t , so $K_{\text{lin}}^{(t)} = K^{(0)}$ exactly, for all t . Conversely, a fixed NTK implies that the network equals its linearization throughout training. The dynamics of the linearized network are exactly those of a classical, well-studied statistical method: *kernel regression*. And we can solve it exactly like how we solved overparameterized linear regression in chapter 1. Given MSE loss,

$$\mathcal{L}(\hat{f}(X)) = \|y - \hat{f}(X)\|^2, \quad \nabla_{\hat{f}(X)} \mathcal{L} = -2(y - \hat{f}(X)), \quad (15)$$

it will be convenient to again track the residual $r^{(t)} := y - \hat{f}_{\text{lin}}^{(t)}(X) \in \mathbb{R}^P$. Since the linearized network has a constant Jacobian $J_{\text{lin}}^{(t)} = J^{(0)}$, its kernel is the constant matrix $K^{(0)} = J^{(0)} J^{(0)\top}$, and Equation (10) becomes

$$\dot{\hat{f}}_{\text{lin}}^{(t)}(X) = 2K^{(0)}r^{(t)} \implies \dot{r}^{(t)} = -2K^{(0)}r^{(t)}, \quad (16)$$

⁶In this sense, I lied a little when I said that linear regression is a terrible toy model of neural networks in chapter 1.

which is the same first order homogeneous linear ODE we met in chapter 1, with $XX^\top$ replaced by $K^{(0)}$. Its solution is

$$r^{(t)} = e^{-2K^{(0)}t} r^{(0)}, \quad \text{i.e.,} \quad \hat{f}_{\text{lin}}^{(t)}(X) = y - e^{-2K^{(0)}t} (y - \hat{f}^{(0)}(X)). \quad (17)$$

Equation (17) is the entire training trajectory of the network's predictions on the training set, in closed form, for all $t \geq 0$. Assuming $K^{(0)}$ is positive definite (the analogue of assuming the rows of X are linearly independent, and generically true for wide networks with $M \gg P$), the residual decays exponentially to zero and the network interpolates (i.e., achieves zero loss on) the training data.

Finally, we usually care about predictions on data we have not trained on. Feeding the above into the linearization at an arbitrary test point x and writing $k^\top(x) \in \mathbb{R}^{1 \times P}$ for the row vector with entries $\langle \phi^{(0)}(x), \phi^{(0)}(x_i) \rangle$,

$$\hat{f}_{\text{lin}}^{(t)}(x) = \hat{f}^{(0)}(x) + k^\top(x) (K^{(0)})^{-1} (I_P - e^{-2K^{(0)}t}) (y - \hat{f}^{(0)}(X)), \quad (18)$$

and at convergence,

$$\hat{f}_{\text{lin}}^{(\infty)}(x) = \underbrace{\hat{f}^{(0)}(x)}_{\text{random init. function}} + \underbrace{k^\top(x) (K^{(0)})^{-1} (y - \hat{f}^{(0)}(X))}_{\text{kernel regression on the residual labels}}. \quad (19)$$

Here's the same result again, via a chain rule intuition. Consider an arbitrary data point $x \sim \rho$ drawn from the data distribution. Since $\hat{f}^{(t)}(x)$ depends on t only through $\theta^{(t)}$, the chain rule gives

$$\dot{\hat{f}}^{(t)}(x) = \frac{\partial \hat{f}^{(t)}(x)}{\partial \theta} \dot{\theta}^{(t)} = - \underbrace{\frac{\partial \hat{f}^{(t)}(x)}{\partial \theta} J^{(t)\top}}_{=: k^\top(x)} \nabla_{\hat{f}(X)} \mathcal{L}(\hat{f}^{(t)}(X)), \quad (20)$$

which integrates, exactly as before, to

$$\hat{f}^{(\infty)}(x) = \hat{f}^{(0)}(x) + k^\top(x) (K^{(0)})^{-1} (y - \hat{f}^{(0)}(X)). \quad (21)$$

We'll assume initialization at zero ($\hat{f}^{(0)}(x) = 0$) for a cleaner expression:

$$\hat{f}^{(\infty)}(x) = k^\top(x) (K^{(0)})^{-1} y. \quad (22)$$

Equation (19) is the punchline of the lazy regime. Up to the random function the network happened to be at initialization, an infinitely wide network trained to convergence in the lazy regime performs *kernel regression with the NTK*. Training does not discover features; the features $\phi^{(0)}$ were fixed the moment we sampled $\theta^{(0)}$, and gradient descent merely finds the least-squares fit in that frozen feature space.

One last observation that we will lean on heavily when we discuss generalization: since $K^{(0)}$ is symmetric PSD, we may diagonalize it as $K^{(0)} = \sum_i \lambda_i v_i v_i^\top$ and decompose the residual along its eigenbasis. Each component then decays independently,

$$\langle v_i, r^{(t)} \rangle = e^{-2\lambda_i t} \langle v_i, r^{(0)} \rangle, \quad (23)$$

so the network learns the top eigendirections of its kernel first and the bottom ones exponentially more slowly. The NTK spectrum tells us, before training begins, which functions the network finds easy and which it finds hard.

But before we go on to study kernel regression, let's return to the problem at hand: showing that $1/\sqrt{H}$ scaling actually yields lazy training.

1.2 $1/\sqrt{H}$ scaling yields lazy training

Let us compute the NTK for the network we introduced earlier in the chapter, $\hat{f}(x) = \frac{1}{\sqrt{H}} \sum_{i=1}^H u_i \sigma(w_i^\top x)$, whose weights are $\theta = \{u_i, w_i\}_{i=1}^H$. The two derivatives we need are

$$\frac{\partial \hat{f}(x)}{\partial u_k} = \frac{1}{\sqrt{H}} \sigma(w_k^\top x), \quad \frac{\partial \hat{f}(x)}{\partial w_k} = \frac{1}{\sqrt{H}} u_k \sigma'(w_k^\top x) x, \quad (24)$$

and summing their products over all weights gives

$$K_{ij} = \underbrace{\frac{1}{H} \sum_{k=1}^H \sigma(w_k^\top x_i) \sigma(w_k^\top x_j)}_{\text{from the output weights}} + \underbrace{(x_i^\top x_j) \frac{1}{H} \sum_{k=1}^H u_k^2 \sigma'(w_k^\top x_i) \sigma'(w_k^\top x_j)}_{\text{from the hidden weights}}. \quad (25)$$

Look at the scaling. The kernel is a sum of H i.i.d. terms each carrying a $\frac{1}{H}$ prefactor. That's LLN scaling. The two derivatives each contributed a factor of $\frac{1}{\sqrt{H}}$, and their product turned the CLT-scaled sum into an LLN-scaled average. Thus, the law of large numbers applies to Equation (25), and as $H \rightarrow \infty$ the random matrix $K^{(0)}$ converges to a deterministic limit:

$$K(x, x') = \mathbb{E}_w [\sigma(w^\top x) \sigma(w^\top x')] + (x^\top x') \mathbb{E}_w [\sigma'(w^\top x) \sigma'(w^\top x')], \quad (26)$$

where we used $\mathbb{E}[u_i^2] = 1$ and the independence of u_i from w_i at initialization. This object is a deterministic function of the architecture, the nonlinearity, and the initialization distribution, with no dependence on any particular draw of the weights. This object is the *analytical* NTK⁷. The width shows up only in how far K strays from it, and the CLT tells us those fluctuations are $\mathcal{O}(1/\sqrt{H})$. In the infinite-width limit, the empirical kernel $K^{(0)}$ is a finite, random sample of the analytical kernel K , where the randomness comes from the fact that the training data $\mathcal{D}$ is a random sample of the data distribution ρ .

Concentration at initialization is only half of what we need; the kernel could still drift once training starts. The following is an intuitive explanation for why it doesn't. Under CLT scaling, the gradient of the loss with respect to any single weight carries a factor of $\frac{1}{\sqrt{H}}$, so over an $\mathcal{O}(1)$ amount of training time each individual weight moves by only $\mathcal{O}(1/\sqrt{H})$. Since the weights themselves are $\mathcal{O}(1)$ at initialization, this is a vanishing relative change, i.e., the weights barely move, which is what “lazy” means. Yet the function moves plenty. The network sums H terms each with a $\frac{1}{\sqrt{H}}$ prefactor, so H weights each budging by $\mathcal{O}(1/\sqrt{H})$ conspire to change $\hat{f}$ by $\frac{1}{\sqrt{H}} \cdot H \cdot \frac{1}{\sqrt{H}} = \mathcal{O}(1)$. This is the resolution of the apparent paradox of lazy training: the

⁷Also called the limiting or infinite-width NTK. Terminology in the literature is not consistent: “the NTK” sometimes means K^∞ and sometimes means $K^{(t)}$ at finite width. We will always write $K^{(t)}$ for the empirical kernel and K for the analytical one.

network learns even though no individual weight does anything appreciable. And the kernel, being a smooth average of quantities that each shift by $\mathcal{O}(1/\sqrt{H})$, inherits that same $\mathcal{O}(1/\sqrt{H})$ deviation rather than an $\mathcal{O}(1)$ one. Concretely,

$$\sup_{t \geq 0} \|K^{(t)} - K^{(0)}\|_2 = \mathcal{O}\left(\frac{1}{\sqrt{H}}\right). \quad (27)$$

In the $H \rightarrow \infty$ limit, since $K^{(t)} = K^{(0)}$ exactly, it's clear that $\hat{f}^{(t)}(X) = \hat{f}_{\text{lin}}^{(t)}(X)$ should be true for all t . However, a stronger claim is true as the fact that the kernel only moves an $\mathcal{O}(\frac{1}{\sqrt{H}})$ amount can be leveraged to show that the network's predictions also only deviate an $\mathcal{O}(\frac{1}{\sqrt{H}})$ amount from the predictions of its linearization:

$$\sup_{t \geq 0} \|\hat{f}^{(t)}(X) - \hat{f}_{\text{lin}}^{(t)}(X)\|_2 = \mathcal{O}\left(\frac{1}{\sqrt{H}}\right). \quad (28)$$

The handwavey intuition given in the past few paragraphs can be made rigorous as mathematical proof. Lee et al. (2019)⁸ do exactly that in appendix G of their paper titled "Wide Neural Networks of Any Depth Evolve as Linear Models Under Gradient Descent." But for our purposes, the handwavey intuition shall suffice.

The technical upshot of this section is that with $\frac{1}{\sqrt{H}}$ scaling and standard Gaussian initialization of the weights, the neural tangent kernel stays fixed throughout training. A fixed NTK forces the neural network to evolve linearly with respect to its parameters, and this simple linear evolution is linear regression in some fixed projection-space—which means it's exactly solvable.

2 Eigenlearning

In this section, we will learn about *Eigenlearning*: a theoretical framework for understanding *generalization* in kernel regression. Generalization is an important term that we haven't talked about much. Let's talk about it.

2.1 On generalization and inductive biases

Recall the setup for a standard supervised learning task. We are given training data consisting of P pairs of data points $\{(x_i, y_i)\}_{i=1}^P$ where the inputs x are sampled (usually assumed i.i.d.) from some data distribution $x \sim \rho$, and the outputs y are generated by some target function plus irreducible noise⁹ $y = f^*(x) + \varepsilon$ (we usually assume $\varepsilon \sim N(0, \sigma^2)$). We want to learn a model $\hat{f}(x)$ that 1) ignores the noise— $\hat{f}(x_i) \approx f^*(x_i)$ for all $i = 1, \dots, P$ —and 2) agrees with $f^*(x)$ on the entire data distribution ρ . These two concerns regarding generalization are largely distinct. The first concern has to do with the fact that the training labels contain irreducible noise. This means that simply driving training error to zero is not necessarily the goal, since we don't want the model to memorize noise. Ideally, the model should extract just the target function $f^*(x)$ and ignore the noise. The second concern has to do with the fact that even if we restrict to functions that get the noise-free targets right on the training set— $\hat{f}(x_i) \approx f^*(x_i)$ for all $i = 1, \dots, P$ —there are enormously many such functions, and only a vanishing fraction of them agree with f^* off the training set. The canonical terms for these two complications are *variance* and *bias*.

⁸<https://arxiv.org/pdf/1902.06720>

⁹We didn't consider irreducible noise in chapter 1, but it will be useful to consider it here.

The bias-variance decomposition

Fix a test point $x \sim \rho$, drawn independently of the training set $\mathcal{D}$ (recall $\mathcal{D}$ bundles both the sampled inputs $x_1, \dots, x_P$ and the sampled noise $\varepsilon_1, \dots, \varepsilon_P$). The learned predictor $\hat{f}$ is a random function of $\mathcal{D}$: different draws of $\mathcal{D}$ give different $\hat{f}$. Define the *expected predictor*

$$\bar{f}(x) \equiv \mathbb{E}_{\mathcal{D}}[\hat{f}(x)],$$

i.e. the function you would get by retraining on infinitely many independent datasets and averaging the resulting predictions pointwise.

Adding and subtracting $\bar{f}(x)$ inside the squared test error and taking $\mathbb{E}_{\mathcal{D}}$, the cross term vanishes exactly. Averaging that pointwise split over $x \sim \rho$ and adding back the irreducible label noise σ^2 (independent of both x and $\mathcal{D}$) then gives an exact decomposition of the test risk:

$$\mathcal{E}(f) \equiv \mathbb{E}_{\mathcal{D}} \left[\mathbb{E}_{x \sim \rho} [(f(x) - \hat{f}(x))^2] \right] + \sigma^2 = \underbrace{\|f - \bar{f}\|^2 + \sigma^2}_{\text{bias } B(f)} + \underbrace{\mathbb{E}_{\mathcal{D}} [\|\hat{f} - \bar{f}\|^2]}_{\text{variance } V(f)}, \quad (29)$$

where $\|g\|^2 \equiv \mathbb{E}_{x \sim \rho} [g(x)^2]$. Working through this split step by step is left as an exercise for the reader.

$B(f)$ is what’s left over even after averaging away all training-set randomness — it measures whether the model’s inductive bias is well matched to f^* , i.e. exactly our second complication (agreement with f^* on all of $\mathcal{D}$). $V(f)$ measures how much $\hat{f}$ fluctuates from one draw of $\mathcal{D}$ (and one noise realization) to another — i.e. how sensitive the learned function is to the particular noise it happened to see, which is exactly our first complication (memorizing noise), restated in decision-theoretic language. A model can have low bias and high variance (it fits a spurious-but-noise-sensitive pattern that happens to average out to f^*), or high bias and low variance (it robustly ignores noise but converges to the wrong function) — so the two really are separate failure modes, not two names for the same thing.

On the “bias-variance tradeoff is outdated” take. What’s often called outdated isn’t the decomposition above — that’s an exact algebraic identity, true unconditionally (Exercise 3.1). What’s outdated is the classical *monotonic* picture layered on top of it: the textbook story that as model capacity grows, bias falls and variance rises monotonically, producing a single U-shaped risk curve with one sweet spot in the middle. Double descent is precisely a violation of that monotonicity, not of the decomposition itself — variance can *spike* at some intermediate capacity and then *fall* again past the interpolation threshold, all while bias keeps monotonically improving. We’ll see this exactly when we get to the eigenlearning framework: test risk splits as $\mathcal{E}(f) = B(f) + V(f)$ with $V(f) = \frac{E_0 - 1}{E_0} \mathcal{E}(f)$, where $E_0 \geq 1$ is an “overfitting coefficient” that can blow up non-monotonically at spectral cutoffs in the kernel — giving a rigorous, closed-form account of exactly the kind of non-monotonic variance behavior that broke the classical intuition.

The solution to both of the above concerns is to consider using a combination of architecture, learning rule, and loss that has an *inductive bias*¹⁰ toward learning a simple, well-generalizing function. While characterizing the inductive biases of neural networks in general is incredibly difficult, doing so for networks specifically in the lazy regime is not, as it directly relates to the frozen NTK determined at initialization. We’ll find that different architectures have different inductive biases that determine how easy or hard it is to learn a particular target function. We’ll also learn about once mysterious deep learning phenomena such as double descent and benign overfitting.

¹⁰The min-norm bias of overparameterized linear regression from chapter 1 and the greedy (larger singular value modes learned first) bias of deep linear networks from chapter 2 are simple examples of inductive biases.

2.2 The crux of eigenlearning

In general, the connection between performance on the training set and performance on the entire data distribution (i.e., performance on a test set in practice) is, mathematically speaking, highly nontrivial. Kernel regression is a special case. In the case of kernel regression, the insight comes from math; specifically, Mercer's theorem. It provides a mathematical link between the finite, training-set-dependent empirical NTK ($K^{(0)}$) and the infinite-dimensional, functional decomposition of the analytical NTK (K). This connection allows us to reason about generalization performance using performance during training. Another way to think about this is that kernel regression has an obvious, mathematically tractable inductive bias.

Because we have to deal with functions (i.e., infinite-dimensional vectors) in addition to finite-dimensional vectors, we will have to rely on some tools of functional analysis. That being said, we will not—and need not—dwell on the mathematical intricacies. For the most part, finite-dimensional linear algebraic intuition will suffice for our purposes.

Suppose we sample N points $\{x_1, \dots, x_N\}$ from our data distribution ρ . We can then create a N -dimensional vector $\hat{f}_N$ where the i th element is $\hat{f}^{(\infty)}(x_i)$. We can think of $\hat{f}_N$ as a finite shadow of the function $\hat{f}$. Doing the same for the target function $f^*(x)$, we get another N -dimensional vector f_N^* . With these finite vectors, we can compute inner products:

$$\frac{1}{N} \langle f_N^*, \hat{f}_N \rangle = \frac{1}{N} \sum_{i=1}^N f^{*(\infty)}(x_i) \hat{f}^{(\infty)}(x_i) \quad (30)$$

where the normalization factor $\frac{1}{N}$ prevents the expression from blowing up as N gets large. By the law of large numbers, as $N \rightarrow \infty$, this expression converges to a fixed quantity: the inner product between the functions f^* and $\hat{f}$.

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N f^{*(\infty)}(x_i) \hat{f}^{(\infty)}(x_i) = \mathbb{E}_{x \sim \rho} [f^*(x) \hat{f}(x)] =: \langle f^*, \hat{f} \rangle. \quad (31)$$

This is how we quantify the "similarity" between functions, and it will serve a pivotal role in our study of kernel regression.

As mentioned in the introduction to the subsection, Mercer's theorem is the mathematical bridge between the empirical and analytical NTK that allows us to reason about test set performance using training set performance. The theorem gives the following functional spectral decomposition of the analytical kernel:

$$K(x, x') = \sum_{i=1}^{\infty} \lambda_i \phi_i(x) \phi_i(x') \quad (32)$$

where the ϕ_i 's are called *eigenfunctions* and the λ_i s are eigenvalues. The eigenfunctions ϕ_i are orthonormal: $\langle \phi_i, \phi_j \rangle = \delta_{ij}$. Analogous to the target SVD basis in chapter 2, the eigenbasis of the analytical kernel is the natural basis to work in to capture generalization in kernel regression. They are entirely deterministic functions, independent of any stochasticity in the sampling of the training data or in the initialization of the weights. We can decompose both $\hat{f}$ and f^* in terms of the eigenbasis:

$$\hat{f}(x) = \sum_{i=1}^{\infty} \hat{v}_i \phi_i(x), \quad f^*(x) = \sum_{i=1}^{\infty} v_i^* \phi_i(x) \quad (33)$$

where $\hat{f}(x)$ indicates the model at the end of training (see Equation (19)). Recall that we know exactly what $\hat{f}(x)$ is going to look like as a result of training since its NTK is frozen. Thus, we need not think about how $\hat{f}(x)$ changes throughout training since we know exactly what it's going to become. Framed this way, it's clear that the learning problem is to get the empirical coefficients $\hat{v}_i$ to well-approximate the analytical coefficients v_i^* .

Recall that the empirical kernel is a random, finite sample of the analytical kernel:

$$K_{ij}^{(0)} = K(x_i, x_j) = \sum_{i=1}^{\infty} \lambda_i \phi_i(x_i) \phi_i(x_j). \quad (34)$$

This means that the "right" way to decompose $K^{(0)}$ is as follows:

$$K^{(0)} = \Phi^\top \Lambda \Phi \quad (35)$$

$$= \underbrace{\begin{bmatrix} -\phi(x_1)^\top - \\ -\phi(x_2)^\top - \\ \vdots \\ -\phi(x_n)^\top - \end{bmatrix}}_{\Phi^\top, n \times \infty} \underbrace{\begin{bmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \lambda_3 & \\ & & & \ddots \end{bmatrix}}_{\Lambda, \infty \times \infty} \underbrace{\begin{bmatrix} | & | & & | \\ \phi(x_1) & \phi(x_2) & \cdots & \phi(x_n) \\ | & | & & | \end{bmatrix}}_{\Phi, \infty \times n}. \quad (36)$$

However, since dealing with infinite-dimensional matrices and vectors introduces many subtle mathematical considerations, we will truncate the eigenmodes at some large but finite number M :

$$K_{ij}^{(0)} \approx \sum_{i=1}^M \lambda_i \phi_i(x_i) \phi_i(x_j) = \underbrace{\tilde{\Phi}^\top}_{n \times M} \underbrace{\tilde{\Lambda}}_{M \times M} \underbrace{\tilde{\Phi}}_{M \times n}. \quad (37)$$

So long as M is sufficiently large, we don't lose anything in discarding the exceedingly small eigenvalue tail. As always, this claim will be validated by the predictions it enables and their match to empirics.

Consider an arbitrary data point $x \sim \rho$ drawn from the data distribution. We worked out the prediction $\hat{f}(x)$ that results from training in Section 1.1.1 (Equation (22)) (assuming initialization at zero for a cleaner expression):

$$\hat{f}(x) = k^\top(x) (K^{(0)})^{-1} y. \quad (38)$$

Since the eigenfunctions are orthonormal (we assume distinct eigenvalues),

$$\hat{v}_i = \langle \phi_i, \hat{f} \rangle = \mathbb{E}_{x \sim \rho} [\phi_i(x) k^\top(x) (K^{(0)})^{-1} \Phi^\top v^*] = \lambda_i \Phi_i^\top (\Phi^\top \Lambda \Phi)^{-1} \Phi^\top v^* \quad (39)$$

$$\hat{v} = \begin{bmatrix} \hat{v}_1 \\ \vdots \\ \hat{v}_M \end{bmatrix} = \Lambda \Phi (\Phi^\top \Lambda \Phi)^{-1} \Phi^\top \begin{bmatrix} v_1^* \\ \vdots \\ v_M^* \end{bmatrix} =: T^{(D)} v^*. \quad (40)$$

where $\Phi_i^\top$ is the i th row of Φ . Let us examine this newly defined matrix $T^{(D)}$, hereon referred to as the *learning transfer matrix*. *TOUCH UP* it's an orthogonal projection matrix (symmetric+idempotent) w.r.t. the RKHS norm ($T^{(D)} = \Lambda^{1/2} \Psi (\Psi^\top \Psi)^{-1} \Psi^\top \Lambda^{-1/2}$ for $\Psi = \Lambda^{1/2} \Phi$) and thus has trace n .

The crux of eigenlearning is that the matrix $T^{(D)}$ is, in expectation with respect to random draws of $\mathcal{D}$, incredibly simple: it's diagonal with the i th diagonal element equaling $\frac{\lambda_i}{\lambda_i + \kappa} =: l_i$ where κ satisfies:

$$n = \sum_{i=1}^{\infty} \frac{\lambda_i}{\lambda_i + \kappa} \quad \left(= \text{tr}(\mathbb{E}_{\mathcal{D}}[T^{(D)}]) \right) \quad (41)$$

(λ_i 's are the eigenvalues of the analytical kernel K). Determining that $T^{(D)}$ is diagonal requires some fairly involved matrix algebra. And showing that the values on the diagonal concentrate into the simple form of $\frac{\lambda_i}{\lambda_i + \kappa}$ requires even more involved random matrix theory. However, the core idea being leveraged is simple; it's an assumption about what the projection matrix Φ looks like. Specifically, the assumption is *Gaussian universality*: $\Phi_{ij} \sim \mathcal{N}(0, 1)$. Of course, the actual distribution of Φ is highly structured, reflecting the eigenstructure of the kernel. The Gaussian universality assumption can be thought of as an active disregard of this fact. There exists some mathematically complex random matrix theory intuition for why this assumption should be true, but this is well beyond the scope of this course. For our purposes, like any assumption in learning mechanics, Gaussian universality is ultimately justified by the close match between the theory it generates and experiments.

The quantity $l_i = \frac{\lambda_i}{\lambda_i + \kappa}$ is the expected *learnability* of the i th eigenmode of the kernel and it equals $\mathbb{E}_{\mathcal{D}}[\langle \phi_i, \hat{f} \rangle]$. In general, for any function f of unit norm ($\|f\| = \sqrt{\langle f, f \rangle} = 1$), its inner product with $\hat{f}$, $\langle f, \hat{f} \rangle =: l_i^{(D)}$ is its $\mathcal{D}$ -learnability. The learnability of a function f is a quantity between 0 and 1 that dictates how well our predictor $\hat{f}$ will learn it.

Concretely, the fact that $T^{(D)}$ is, in expectation, diagonal with the i th diagonal element equaling $\frac{\lambda_i}{\lambda_i + \kappa}$ means that

$$\mathbb{E}_{\mathcal{D}}[T_{ij}^{(D)}] = \frac{\lambda_i}{\lambda_i + \kappa} \delta_{ij} \implies \mathbb{E}_{\mathcal{D}}[\hat{v}_i] = \frac{\lambda_i}{\lambda_i + \kappa} v_i^*. \quad (42)$$

2.3 An equation for generalization

Recall our expression for the test loss:

$$\mathcal{E}(f^*) \equiv \mathbb{E}_{\mathcal{D}} \left[\mathbb{E}_{x \sim \rho} [(f^*(x) - \hat{f}(x))^2] \right] + \sigma^2. \quad (43)$$

Dealing with the irreducible noise term σ^2 above requires more care than one might expect since the predictor $\hat{f}$ has a subtle dependence on the noise that was part of the training data $\mathcal{D}$. Intuitively, we will think about the noise as being an *unlearnable* part of the target function f^* . We'll see mathematically what this means later. For now, what matters is that we can disregard the σ^2 term and think of it as being embedded somehow in the $\mathbb{E}_{\mathcal{D}} \left[\mathbb{E}_{x \sim \rho} [(f^*(x) - \hat{f}(x))^2] \right]$ term.

By orthonormality of the eigenfunctions, we can rewrite Equation (43) in terms of the empirical and analytical coefficients $\hat{v}_i$ and v_i^* :

$$\mathcal{E}(f^*) = \mathbb{E}_{\mathcal{D}} \left[\sum_i (v_i^* - \hat{v}_i)^2 \right]. \quad (44)$$

Propagating the expectation and using the fact that $\hat{v} = T^{(D)} v^*$,

$$\mathcal{E}(f^*) = \mathbb{E}_{\mathcal{D}} \left[\sum_i (v_i^* - \hat{v}_i)^2 \right] \quad (45)$$

$$= \mathbb{E} \left[v^{*\top} (I - T^{(D)})^2 v^* \right] \quad (46)$$

$$= v^{*\top} \left(\mathbb{E}_{\mathcal{D}} [T^{(D)^2}] - 2\text{diag}(l_i) + I \right) v^*. \quad (47)$$

The tricky term above is $\mathbb{E}_{\mathcal{D}} [T^{(D)^2}]$. Computing it requires a decent amount of matrix algebra and random matrix theory that we will not go over. The relevant result is:

$$\mathbb{E}_{\mathcal{D}} [T_{ij}^{(D)} T_{ik}^{(D)}] = \frac{l_i^2(1-l_j)^2}{n - \sum_m l_m^2} \delta_{jk} + l_i^2 \delta_{ij} \delta_{ik}. \quad (48)$$

Plugging this in and simplifying, the j, k sums collapse. Writing $X \equiv \sum_i (1-l_i)^2 v_i^{*2}$, the $\delta_{ij} \delta_{ik}$ term contributes X and the δ_{jk} term contributes $(\sum_m l_m^2)X/(n - \sum_m l_m^2)$, so that

$$\mathbb{E}_{\mathcal{D}} \left[\sum_i (v_i^* - \hat{v}_i)^2 \right] = \left[1 + \frac{\sum_m l_m^2}{n - \sum_m l_m^2} \right] X = \underbrace{\frac{n}{n - \sum_m l_m^2}}_{\equiv \mathcal{E}_0} \sum_i (1-l_i)^2 v_i^{*2}. \quad (49)$$

2.3.1 Absorbing the label noise into the target

It remains to deal with the irreducible noise in the test loss. Naively one might simply add it back on at the end, but this would be wrong: the noise does not only appear at test time, it is also present in the training labels $y_i = f^*(x_i) + \varepsilon_i$, and $\hat{f}$ is a linear function of y . The estimator therefore fits some of the training noise, and that fitted noise shows up in the test error as well.

The clean way to handle both effects at once is to note that the noise enters the problem in exactly the same way a target function component would, and to treat it as such. Formally, augment the eigenbasis with a set of M' additional orthonormal functions $\{\chi_a\}$ with eigenvalues $\lambda_\chi = 0$, and write the noise as a target component supported on them,

$$f_{\text{aug}}^*(x) = \sum_i v_i^* \phi_i(x) + \sum_{a=1}^{M'} u_a \chi_a(x), \quad \sum_a u_a^2 = \sigma^2. \quad (50)$$

Three facts make this substitution legitimate:

1. **It reproduces the correct label statistics.** The ε_i are i.i.d., mean zero, with variance σ^2 , and are uncorrelated with x . A target component spread across directions the kernel cannot resolve has exactly these properties as seen by the estimator: it contributes σ^2 of label variance, uncorrelated with anything in the kernel's span.
2. **The energy is basis-independent.** White noise has no preferred basis, so its total power σ^2 is the same however it is decomposed. Taking $M' \rightarrow \infty$, each individual $u_a^2 \rightarrow 0$ while $\sum_a u_a^2 = \sigma^2$ is exactly preserved. Nothing is created or destroyed by the change of description.
3. **Zero eigenvalue is the right assignment.** From $l = \lambda/(\lambda + \kappa)$, $\lambda_\chi = 0$ gives $l_\chi = 0$, i.e. $\mathbb{E}[\hat{u}_a] = l_\chi u_a = 0$: the estimator has no systematic component along these directions, matching the fact that zero-mean noise cannot bias the predictor. Note also that adding modes with $\lambda = 0$ leaves Eq. (6) untouched, so κ , and hence every l_i , is unchanged.

With this substitution the noise is no longer a separate object: it is simply part of the target, and the result derived above applies verbatim. Since $l_\chi = 0$ implies $(1 - l_\chi)^2 = 1$, the noise directions enter the sum undiscounted:

$$\sum_i (1 - l_i)^2 v_i^{*2} \longrightarrow \sum_i (1 - l_i)^2 v_i^{*2} + \sum_a \underbrace{(1 - l_\chi)^2}_{=1} u_a^2 = \sum_i (1 - l_i)^2 v_i^{*2} + \sigma^2, \quad (51)$$

and likewise $\sum_m l_m^2$ is unchanged, since $l_\chi^2 = 0$. Hence $\mathcal{E}_0$ is unchanged, and

$$\boxed{\mathcal{E}(f^*) = \mathcal{E}_0 \left(\sum_i (1 - l_i)^2 v_i^{*2} + \sigma^2 \right), \quad \mathcal{E}_0 = \frac{n}{n - \sum_m l_m^2}} \quad (52)$$

Why $\sigma^2 \rightarrow \mathcal{E}_0 \sigma^2$ is not double counting

The appearance of $\mathcal{E}_0$ multiplying σ^2 deserves comment, since σ^2 is often described as an “irreducible” noise floor and it is not obvious why an irreducible quantity should be amplified. It is not: the product is the sum of two distinct contributions,

$$\mathcal{E}_0 \sigma^2 = \underbrace{1 \cdot \sigma^2}_{\text{test-label noise}} + \underbrace{(\mathcal{E}_0 - 1) \sigma^2}_{\text{fitted training noise}}. \quad (53)$$

The first term is the genuinely irreducible piece: even a perfect predictor $\hat{f} = f^*$ is scored against noisy test labels and incurs σ^2 . It enters with coefficient exactly one. The second term is variance, and is the price of having fit n noisy training labels: it is the noise contribution to $\mathcal{V}(f^*) = (\mathcal{E}_0 - 1)\mathcal{B}(f^*)$. In the ridgeless, well-specified limit $\mathcal{E}_0 \rightarrow 1$ the second term vanishes and only the true floor survives, as it must.

It is worth being precise about the mechanism behind the second term, since $l_\chi = 0$ might suggest the estimator ignores the noise directions entirely. What $l_\chi = 0$ states is that $\mathbb{E}_{\mathcal{D}}[\hat{u}_a] = 0$: averaged over draws of $\mathcal{D}$, the predictor has no component along χ_a . It does not state that $\hat{u}_a = 0$ for any particular $\mathcal{D}$. The off-diagonal entries of $T^{(\mathcal{D})}$ have vanishing mean but non-vanishing covariance, which is precisely the content of the δ_{jk} term in Eq. (86); on any given training set the fit absorbs a nonzero sliver of the noise, and these slivers do not cancel within a single draw. Their aggregate is exactly $(\mathcal{E}_0 - 1)\sigma^2$.

Finally, note that $\mathcal{E}_0$ is not itself a noise-induced quantity. Setting $\sigma^2 = 0$ leaves $\mathcal{E}(f^*) = \mathcal{E}_0 \sum_i (1 - l_i)^2 v_i^{*2}$ with $\mathcal{E}_0 > 1$ in general: the overfitting coefficient is a statement about finite-sample fluctuations of the estimator, and applies identically to signal and noise. The noise-as-target construction explains why σ^2 sits *inside* the bracket; the factor multiplying that bracket was already there.

2.4 Double descent and benign overfitting

We now have everything we need to explain two phenomena that, for a while, looked like genuine paradoxes: *double descent* (test error that gets worse, then better, as we add data or capacity) and *benign overfitting* (a model that interpolates noisy training labels exactly and yet generalizes fine). The eigenlearning equations say that both live in a single scalar,

$$\mathcal{E}_0 = \frac{n}{n - \sum_m l_m^2}, \quad (54)$$

and in particular in how close the sum $\sum_m l_m^2$ gets to n . Our whole job in this subsection is to understand what makes that happen.

The learnability budget

Start with the equation that defines κ :

$$\sum_m l_m = \sum_m \frac{\lambda_m}{\lambda_m + \kappa} = n. \quad (55)$$

This is worth staring at. The total learnability summed over all eigenmodes is *exactly* n , no matter what the kernel is. Each training point buys one unit of learnability, and κ is the price that adjusts so that the books balance. Think of κ as a water level: modes with $\lambda_m \gg \kappa$ stick out above the water and are essentially fully learned ($l_m \approx 1$), modes with $\lambda_m \ll \kappa$ are submerged and essentially unlearned ($l_m \approx 0$), and modes with $\lambda_m \sim \kappa$ are partially learned. Adding data lowers the water level, exposing more modes.

The kernel does not get to choose how much learnability it has; it only gets to choose *how to spread it*. And $\sum_m l_m^2$ is precisely a measure of how concentrated that spreading is. Since every $l_m \in [0, 1]$ we have $l_m^2 \leq l_m$, so

$$\sum_m l_m^2 \leq \sum_m l_m = n, \quad (56)$$

with equality exactly when every l_m is either 0 or 1. So $\mathcal{E}_0$ blows up precisely in the all-or-nothing case: when the training set is spent on n modes that are each learned completely, and nothing is left over.

It is worth making this quantitative. Define the *participation ratio*

$$n_{\text{eff}} \equiv \frac{(\sum_m l_m)^2}{\sum_m l_m^2} = \frac{n^2}{\sum_m l_m^2}, \quad (57)$$

the standard way of asking “how many modes is this fit actually spread over?”¹¹ Substituting, the overfitting coefficient becomes

$$\boxed{\mathcal{E}_0 = \frac{n_{\text{eff}}}{n_{\text{eff}} - n}, \quad \mathcal{E}_0 - 1 = \frac{n}{n_{\text{eff}} - n}.} \quad (58)$$

This is the mechanistic statement we were after, and it should look extremely familiar. It is the classical variance-inflation factor of a linear regression with n_{eff} parameters fit to n data points. The kernel’s eigenspectrum, together with the amount of data, determines an *effective number of parameters* n_{eff} , and everything hinges on the slack $n_{\text{eff}} - n$:

- $n_{\text{eff}} \gg n$: lots of slack. The fit is spread across far more directions than there are constraints, so each direction only has to absorb a sliver of the training noise. $\mathcal{E}_0 \approx 1$, and almost none of the fitted noise makes it into the test error.
- $n_{\text{eff}} \rightarrow n$: no slack. There are exactly as many usable directions as there are training points, so the fit is fully pinned down by the data, noise and all, with no freedom left to hedge. $\mathcal{E}_0 \rightarrow \infty$.

The $n_{\text{eff}} = n$ point is the *interpolation threshold*, and it is the peak of the double descent curve.

¹¹If k modes have $l_m = 1$ and the rest have $l_m = 0$, then $n_{\text{eff}} = k$. If the learnability is spread evenly and thinly over k modes, $n_{\text{eff}} = k$ again. In both cases it counts the modes the fit is genuinely using.

Double descent: a spectrum with a cliff

To see the peak explicitly, take the simplest possible spectrum: M eigenmodes with equal eigenvalue λ , and nothing else ($\lambda_m = 0$ for $m > M$). This is a kernel with a hard cliff in its spectrum. By symmetry every l_m is equal, and Equation (55) gives $Ml = n$, so $l = n/M$ for each of the M modes. Then $\sum_m l_m^2 = n^2/M$, i.e. $n_{\text{eff}} = M$ — the effective parameter count is just the number of modes, as it should be — and

$$\mathcal{E}_0 = \frac{M}{M - n}. \quad (59)$$

Feeding this into the eigenlearning equations, the test error for a target supported on these modes is

$$\mathcal{E}(f^*) = \frac{M}{M - n} \left[\left(1 - \frac{n}{M}\right)^2 \|f^*\|^2 + \sigma^2 \right] = \underbrace{\left(1 - \frac{n}{M}\right) \|f^*\|^2}_{\text{signal}} + \underbrace{\frac{M}{M - n} \sigma^2}_{\text{noise}}. \quad (60)$$

The two terms behave completely differently. The signal term decreases smoothly and monotonically to zero as $n \rightarrow M$: more data always means more of the target is learned, and the bias never misbehaves. The noise term *diverges* at $n = M$. The familiar U-then-descent shape is a race between these two, and the peak is entirely a noise-amplification effect: at $n = M$ we are asking M degrees of freedom to fit M noisy labels exactly, and the unique solution is forced to contort itself through every one of them.¹²

The lesson generalizes past this toy example. Anything that makes the learnabilities *near-binary* pushes $\sum_m l_m^2$ toward n and drives $\mathcal{E}_0$ up: a sharp cliff in the eigenspectrum, a large gap between eigenvalue scales with κ sitting in the gap, or more generally any spectrum for which the water level κ lands in a region with no modes near it. Double descent is not a mysterious property of neural networks; it is what happens when a spectrum has a cliff and your dataset size walks off it.

Benign overfitting: the tail provides the slack

Now modify the example. Keep the M “head” modes with eigenvalue λ , but add a long tail: a very large number of additional modes with tiny eigenvalues $\varepsilon \ll \lambda$. Real kernels — the NTKs of actual architectures very much included — look like this. Their spectra decay, often as a power law, but they do not stop.

Set $n = M$, exactly where the cliff-only kernel exploded. Because there are so many tail modes, the tail can supply a nonzero total learnability

$$q \equiv \sum_{m \in \text{tail}} l_m > 0 \quad (61)$$

even though each individual l_m in the tail is minuscule. The budget then leaves only $n - q$ for the head, so each head mode gets $l_{\text{head}} = (n - q)/M = 1 - q/M$ rather than the full 1. Now compute the dangerous sum. The head contributes $M(1 - q/M)^2 \approx M - 2q$, and the tail

¹²Past the threshold ($n > M$) the model can no longer interpolate, the ridgeless limit $\kappa \rightarrow 0$ is no longer consistent with Equation (55), and the problem becomes an ordinary overdetermined least-squares fit which *averages* the noise rather than chasing it. The error then falls again — the second descent. Ordinary double descent as usually plotted varies model capacity at fixed n ; here we varied n at fixed capacity M . It is the same peak, approached from the other axis, because only the ratio n/n_{eff} matters.

contributes essentially nothing, since squaring a large number of tiny learnabilities annihilates them ($\sum_{\text{tail}} l_m^2 \ll \sum_{\text{tail}} l_m = q$). So

$$\sum_m l_m^2 \approx n - 2q \quad \implies \quad n_{\text{eff}} \approx n + 2q \quad \implies \quad \mathcal{E}_0 \approx \frac{n}{2q}. \quad (62)$$

The divergence is gone. A tail that is individually negligible — modes so small that they contribute nothing to fitting the target and are invisible in the bias term — has collectively bought $2q$ units of slack in n_{eff} , and that slack is exactly what caps the peak. This is the mechanism of benign overfitting. The model still interpolates the training data, noise included. But the fitted noise gets dumped into a huge number of low-eigenvalue directions, each absorbing a sliver, and because those directions carry almost no weight in the kernel they are almost invisible on the test distribution. Overfitting still happens; it is just harmless, in exactly the sense that $(\mathcal{E}_0 - 1)\sigma^2$ is small.

This also tells us when overfitting stops being benign: when the tail is too short or decays too fast to supply meaningful slack ($q \rightarrow 0$), or when the water level κ sits in a spectral gap so that the modes are cleanly sorted into learned and unlearned with nothing in between.

One knob, two phenomena

It is worth stating plainly what we have found, because the usual presentations of these two phenomena make them sound unrelated:

$$\begin{array}{ll} \textbf{Double descent} & \sum_m l_m^2 \rightarrow n, \text{ i.e. } n_{\text{eff}} \rightarrow n: \text{ no slack, } \mathcal{E}_0 \text{ blows up.} \\ \textbf{Benign overfitting} & \sum_m l_m^2 \ll n, \text{ i.e. } n_{\text{eff}} \gg n: \text{ lots of slack, } \mathcal{E}_0 \approx 1. \end{array}$$

They are the two ends of a single knob, and the knob is the interaction between the kernel’s eigenspectrum and the dataset size. Neither is a statement about depth, nonlinearity, or anything specific to neural networks — they are properties of a spectrum and a value of n . What the NTK buys us is the right to say that a wide network in the lazy regime *has* such a spectrum, fixed at initialization by the architecture, which is why an architecture’s inductive bias and its overfitting behavior turn out to be the same question.

Note also that this resolves the puzzle we flagged back in the bias-variance discussion. Nothing here contradicts the decomposition $\mathcal{E} = B + V$; the bias term is perfectly well-behaved and monotone in the example above. What breaks is only the classical assumption that variance rises monotonically with capacity. Variance is $\frac{\mathcal{E}_0 - 1}{\mathcal{E}_0} \mathcal{E}$, and $\mathcal{E}_0$ is controlled by $n_{\text{eff}} - n$, which is not monotone in anything in particular. It can shrink to zero at a spectral cliff and grow again once the tail is reached.

3 Why lazy learning is bad

At a high level, lazy learning is bad because learning is movement and networks undergoing lazy learning barely move in parameter space.